



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech



OPERATING SYSTEMS

1-JULY-26

Contents

- Introduction to Operating Systems
- Process Management
- Memory Management
- File System Management
- Disk Management
- Linux Commands & Shell Basics

Introduction to Operating Systems

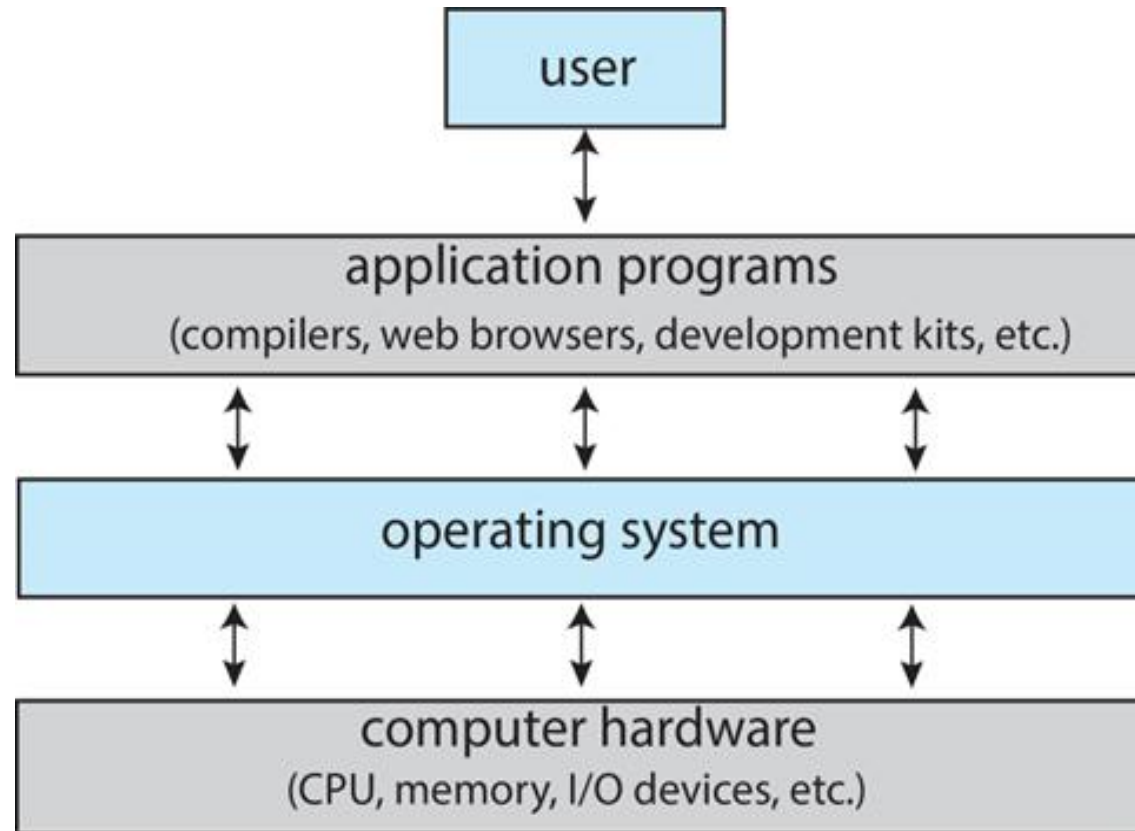


What is Operating System ?

- An **Operating System (OS)** is a **collection of software** that **manages computer hardware resources** and provides **common services for computer programs**.
- The operating system is the most **important type of system software** in a computer system.
- An operating system acts as an **intermediary between the user of a computer and computer hardware**.
- The purpose of an operating system is to provide **an environment** in which a **user** can **execute programs conveniently and efficiently**.

Components of Computer system

The computer system can be divided into **four components**:



Components of Computer system

1. Hardware: Provides the basic computing resources.

Examples: CPU, memory, input/output (I/O) devices.

2. Operating System (OS): Controls and coordinates the use of hardware resources among various application programs and users. It acts as an intermediary between users and the hardware.

Examples : Windows, Linux, macOS, Unix, Android, iOS

3. Application Programs: Define how system resources are used to solve users' computing problems.

Examples: Word processors, web browsers, database systems, video games.

4. Users: Entities that interact with the computer system.

Examples: People, machines, other computers.

Key roles of an Operating System

1. Resource Management

- **CPU Management:** Allocates and deallocates CPU time using scheduling algorithms.
- **Memory Management:** Manages physical and virtual memory, allocates memory to processes, and handles swapping.
- **Device Management:** Controls and manages I/O devices using device drivers.
- **Storage Management:** Manages data storage, file systems, and disk space.

2. Process Management

- Handles creation, execution, and termination of processes.
- Manages **process scheduling**, synchronization, and communication.
- Provides **multitasking** and **multiprocessing** capabilities.

Key roles of an Operating System

3. File System Management

- Manages **files and directories** on storage devices.
- Controls **access, organization, storage, retrieval, naming, sharing, and protection** of files.

4. Security and Protection

- Ensures that **unauthorized users** do not access the system or data.
- Implements **authentication (login), access control, and encryption**.

5. User Interface

- Provides a user interface (**CLI or GUI**) for interaction.
- Examples: **Command Prompt (Windows), Terminal (Linux), or Graphical Desktop Environments**.

Key roles of an Operating System

6. System Performance Monitoring

- Tracks **system performance** and **optimizes** resource usage.
- May include tools to monitor **CPU, memory, disk usage, etc.**

7. Networking

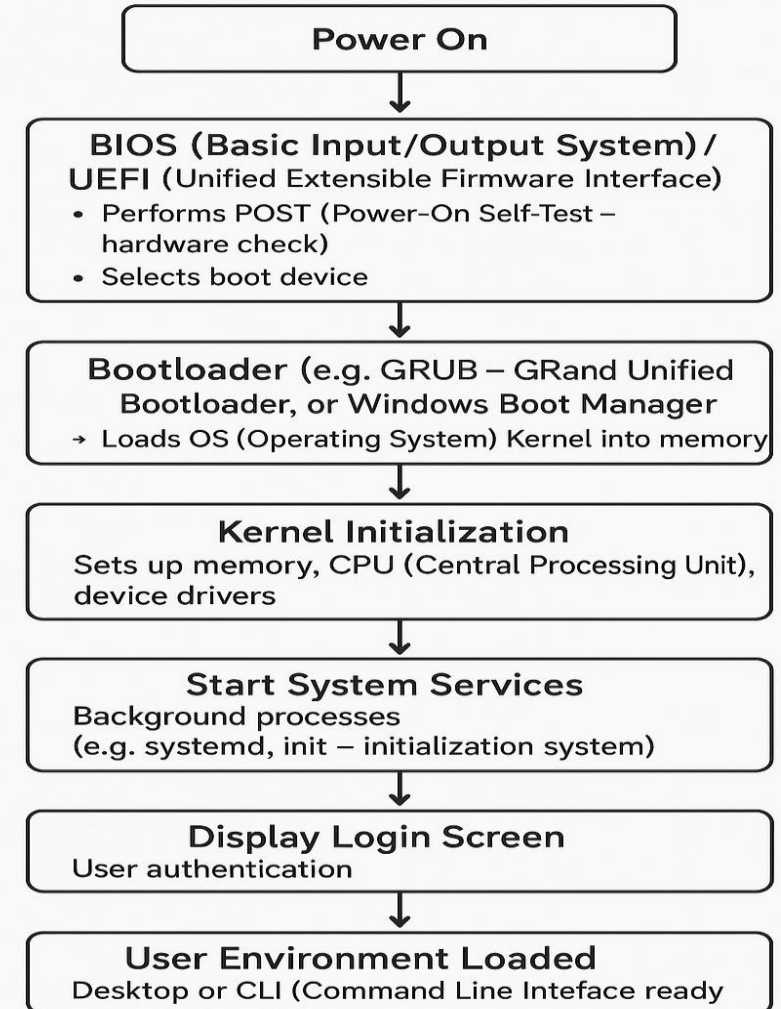
- Manages **network connections and data communication** across systems.
- Supports **protocols, sockets, and internet access.**

8. Error Detection and Handling

- Detects and responds to **hardware and software errors.**
- **Maintains logs** and may **attempt recovery** or alert the user.

OS Boot Process

- The Operating System (OS) boot process is the **sequence of steps** that occur from powering on a computer to **loading the operating system** and becoming ready for user interaction.
- Here are the **main steps in the OS boot process** given in the picture :



OS Concepts

1. Multiprogramming

- Many programs are in memory, but **only one runs at a time**.
- When one program waits (e.g., for file read), the **OS switches to another**.
- **Goal: Keep the CPU busy — no idle time.**
- **Example:**
 - A computer is compiling code, downloading a file, and printing a document.
 - While the printer waits for paper, the CPU works on compiling.

OS Concepts

2. Multitasking

- A user runs **many tasks at once**, and the CPU switches quickly between them.
- Feels like everything is **happening together**.
- Goal: Provide **smooth user experience**.
- **Example:**
 - You're typing a report, listening to music, and browsing the web — all on one computer.
 - The OS gives each task a small time slice so all seem to work at the same time.

OS Concepts

3. Multiprocessing

- The system has **two or more CPUs** (or cores) working **at the same time**.
- Real parallel execution.
- **Goal:** Boost speed and handle more work.
- **Example:**
 - A multi-core laptop can run a video game, antivirus scan, and background updates — each on a different core.

Types of Operating Systems

Operating systems come in various types, each designed to **meet specific needs and environments** — from managing simple appliances to powering massive distributed networks.

Below are the types of Operating Systems :

1. Batch Operating System :

- **No direct interaction with the user.** Jobs (tasks) are collected, grouped into batches, and processed one after another.
- **Key Idea:** Automation — the user submits a **batch of jobs and waits for the output**.
- **Example:** In early banking systems, customer transactions were entered on punch cards, collected into batches, and processed overnight.

Types of Operating Systems

2. Time-Sharing Operating System:

- Many users **use the system at the same time**. The OS quickly switches between **users/tasks**, giving each a small amount of **CPU time**.
- **Key Idea:** Fairness — makes it feel like everyone is using the **system simultaneously**.
- **Example:** College or research lab servers where multiple students log in remotely and run programs or simulations.

3. Distributed Operating System:

- Connects **multiple computers (nodes)** and makes them work together as one system.
- **Key Idea:** Resource Sharing — CPU, memory, files, etc., are **shared across the network**.
- **Example:** Google Search runs on thousands of servers around the world.

Types of Operating Systems

4. Real-Time Operating System (RTOS):

- Used in **time-critical systems** where responses must happen **immediately or within strict deadlines**.
- **Key Idea:** Predictable and fast response.
- **Example:** Airbag system in a car must respond within milliseconds during a crash; otherwise, it fails its purpose.

5. Network Operating System (NOS):

- Designed to **manage networked computers**, enabling file sharing, printer access, security, and user management.
- **Key Idea:** Central control over a network.
- **Example:** Windows, Linux Server used in school/office environments to control all connected computers.

Types of Operating Systems

6. Mobile Operating System:

- Specifically built for **smartphones, tablets, and other handheld devices**.
- **Key Idea:** Touch support, sensors, lightweight apps.
- **Example:** Android on Samsung phones, iOS on iPhones.

7. Embedded Operating System:

- Runs inside **dedicated devices** with a specific function.
- **Key Idea:** Small, efficient, and reliable — doesn't need full desktop features.
- **Example:** A smart washing machine, microwave oven, or digital camera that performs a specific task based on input.

Process Management



Introduction - Process

- A process is a **program in execution**. A process is an '**active**' entity instead of a program, which is considered a '**passive**' entity.

Example:

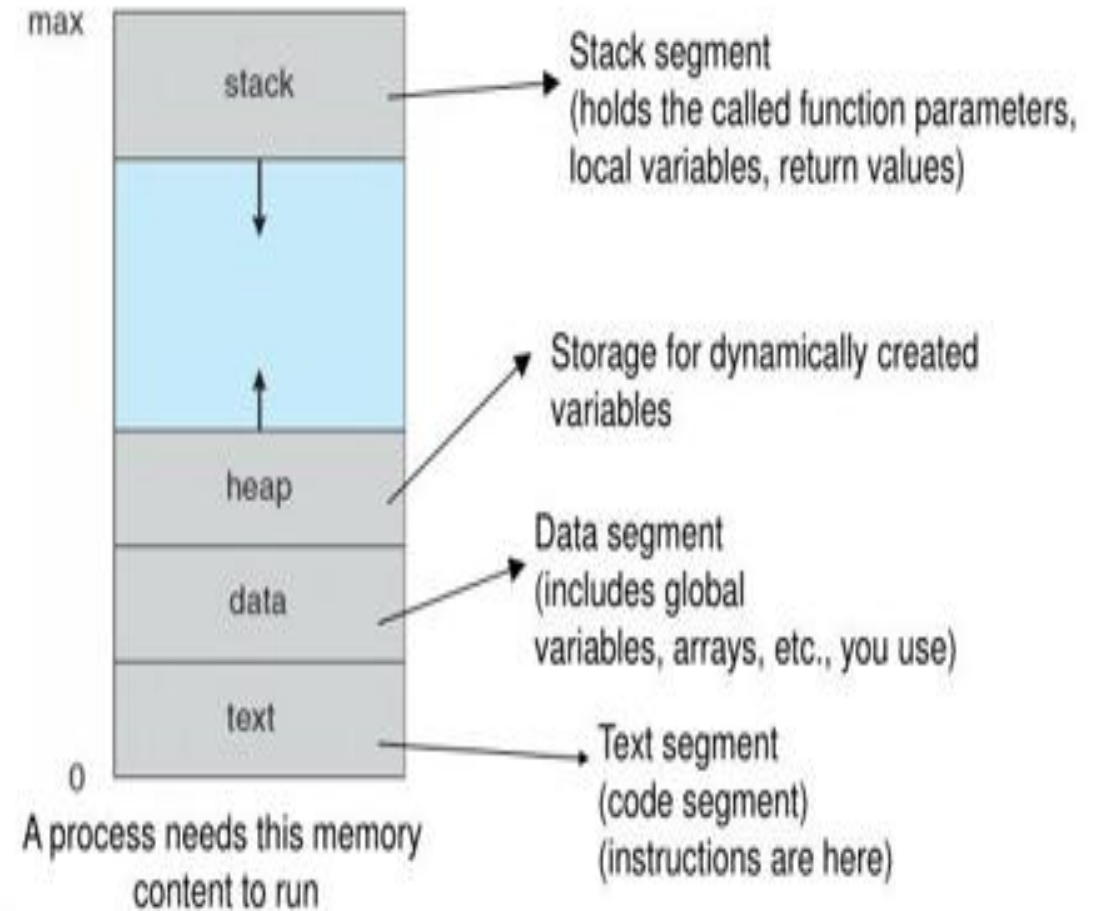
Let's say you open Google Chrome on your computer.

- The Chrome **application** is stored as a program (in your hard drive).
- When you double-click it, it becomes a **process**.
- The OS loads it into **memory, allocates resources**, and starts **running** it.
- If you open 3 tabs, Chrome may run **multiple processes** to handle each tab separately.

Process Management

How does a process look like in Memory?

- When a program is loaded into memory and becomes a **process**, the operating system organizes it into **different sections**.
- Here's the typical layout of a process in memory (from low 0 to max address):



How does a process look like in Memory?

Segment	Purpose	Example
Code Segment	Contains the actual instructions to be executed.	Your compiled C/Java code
Data Segment	Stores global and static variables (initialized or uninitialized).	int x = 10;
Heap	Used for dynamic memory allocation during runtime.	malloc(), new
Stack	Stores function call information, return addresses, local variables.	Recursion, nested function calls

Process states

- A process goes through **various states during its lifetime.**
- **The main states are:**

State	Meaning
New	Process is being created
Ready	Process is waiting to be assigned to the CPU
Running	Process is currently being executed by the CPU
Waiting	Process is waiting for some I/O or event to occur
Terminated	Process has finished execution

Process Control Block

- The Process Control Block (PCB) is a data structure used by the **OS to store all information about a process.**
- **Key Fields of the Process Control Block (PCB)** are given below :

PCB Field	Description
Process ID (PID)	Unique ID of the process
Process State	Current state (Ready, Running, etc.)
Program Counter	Address of the next instruction to execute
CPU Registers	Contents of CPU registers
Memory Info	Base and limit registers, memory map
I/O Info	List of open files, I/O devices used
Accounting Info	CPU usage, time limits, process priority, etc.

What is a Thread?

- A **thread** is the **smallest unit of execution** within a process.
- **Example: Web Browser (like Chrome)** . When you open Chrome:
 - It runs as a process.
 - Inside that process:
 - One thread handles the **user interface (UI)**.
 - Another thread **loads a webpage**.
 - Another thread **downloads a file**.
 - Another plays **background music or video**.
- These threads **run in parallel** within the same process and **share memory**, making them fast and lightweight.

Types of Thread

1. User-Level Threads (ULT) :

- Managed entirely in **user space**
- OS doesn't know **multiple threads exist** inside the process
- **Faster** to create/manage
- **Disadvantage:** If one thread blocks (e.g., for I/O), the entire process blocks
- **Example:** Early versions of Java threads, some scripting

2. Kernel-Level Threads (KLT) :

- Managed by the **OS kernel**
- OS schedules threads **independently**
- Each thread **can run on a different CPU core**

Types of Thread

- **Slower** to manage due to kernel overhead
- **Example:** POSIX Threads (pthreads), Windows threads

3. Hybrid Threads :

- Uses both **ULT and KLT**
- Allows **multiple user threads** to be mapped to multiple kernel threads (M:N model)
- Balances **performance and control**
- **Example:** Solaris, modern Java Virtual Machine (JVM)

Process vs Thread

Process	Thread
A program in execution	A lightweight unit of a process
Has its own memory space	Shares memory space with other threads in process
High overhead (takes more resources)	Low overhead (lightweight)
Slower to create and manage	Faster to create and manage
Inter-process communication (IPC) is complex	Threads can communicate easily (shared memory)
If one process crashes, others are unaffected	If one thread crashes, it may affect the process
Each process has its own stack, data, code, etc.	Threads have their own stack, but share data and code
Example: MS Word and Chrome are separate processes	Example: In Chrome, one thread handles UI, another loads pages

Context Switching

- **Context Switching** is the process where the **CPU switches from one process or thread to another**.
- Since only **one process/thread can run on a CPU core at a time**, the operating system must **save the state** of the currently running process and **load the state** of the next process to run. This switch is called **context switch**.

Context Switching

When Does Context Switching Happen?

1. A process is interrupted (e.g., by a timer or I/O)
2. A higher-priority process becomes ready
3. The running process voluntarily gives up CPU (e.g., waiting for I/O)
4. In multitasking and multiprocessing systems

Note: The OS uses PCB during context switching to save and restore the process's state.

Example: Like a traffic controller deciding which car (process) goes next at a signal (CPU).

CPU Scheduling

- **CPU Scheduling** is the process by which the **Operating System (OS)** decides **which process from the Ready Queue should be assigned to the CPU next**, when multiple processes are waiting to execute.

Need of CPU Scheduling :

- In **multiprogramming systems**, many processes may be ready to run.
- But the **CPU can only run one process at a time** (on a single-core system).
- So, the OS uses CPU scheduling to decide **which process runs, when to switch, and how long each process gets**.

CPU Scheduling

What Happens During CPU Scheduling?

- Multiple processes are in the **Ready Queue**.
- The **CPU Scheduler** picks one based on a **scheduling algorithm**.
- The **selected process runs** on the CPU.
- If it completes, blocks for I/O, or gets preempted, the OS **saves its state** (context switch) and selects another.

CPU Scheduling - Terminologies

Term	Meaning
CPU Burst	The time a process spends using the CPU before doing I/O.
I/O Burst	The time a process spends doing input/output operations.
Throughput	Number of processes completed per unit of time.
Completion Time	The time at which a process finishes execution and leaves the CPU.
Turnaround Time	Total time taken from process submission to completion.
Waiting Time	Total time a process spends in the ready queue, waiting for CPU.
Response Time	Time from process submission to first response (first time it gets the CPU).
Starvation	A process waits indefinitely because other processes are continuously selected over it.
Aging	A technique to prevent starvation by gradually increasing the priority of waiting processes.

CPU Scheduling -Types

CPU Scheduling is broadly categorized into two types:

1. Preemptive Scheduling

- The OS **can interrupt** a running process and assign the CPU to another.
- Useful in **time-sharing systems** where responsiveness is important.
- Improves responsiveness but involves **more context switching**.

Examples of Preemptive Algorithms:

Algorithm	Description
Round Robin (RR)	Each process gets a fixed time slot (quantum).
Preemptive Shortest Job First (SRTF)	The process with the shortest remaining time is selected.
Preemptive Priority Scheduling	CPU given to the highest-priority process, even if another is running.

CPU Scheduling -Types

2. Non-Preemptive Scheduling

- Once a process gets the CPU, it **runs until it completes** or blocks (no interruption).
- Simpler and has **less overhead**, but **less responsive**.

Examples of Non-Preemptive Algorithms:

Algorithm	Description
First Come First Serve (FCFS)	Processes are executed in the order they arrive.
Non-Preemptive SJF	Process with the shortest job is selected first.
Non-Preemptive Priority	Highest-priority process runs to completion before the next is selected.

CPU Scheduling –Gantt Chart

A **Gantt Chart** is a **visual timeline** that shows the **order and duration** of processes being executed by the CPU.

- It helps track **when** a process starts and **when** it ends.
- Useful to **calculate** metrics like:
- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)

CPU Scheduling – Example

- Consider the following set of processes with their Arrival Time (AT) and Burst Time (BT).
- Calculate **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)** using various **CPU scheduling algorithms**.

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

CPU Scheduling – Example : FCFS

1) First Come First Serve (FCFS) :

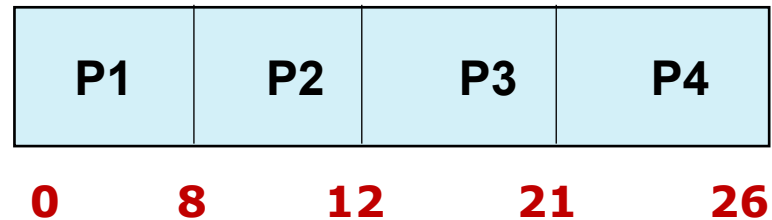
- Processes are executed in the **order they arrive** (first come, first served).
- Non-preemptive**: Once a process starts, it runs to completion.

Step 1: Order by Arrival Time

Execution order: P1 → P2 → P3 → P4

Step 2: Gantt Chart

- P1 starts at 0 → CT = 0 + 8 = **8**
- P2 starts at 8 → CT = 8 + 4 = **12**
- P3 starts at 12 → CT = 12 + 9 = **21**
- P4 starts at 21 → CT = 21 + 5 = **26**



CPU Scheduling – Example : FCFS

Step 3: CT, TAT, WT Calculations :

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$
P2	1	4	12	$12 - 1 = 11$	$11 - 4 = 7$
P3	2	9	21	$21 - 2 = 19$	$19 - 9 = 10$
P4	3	5	26	$26 - 3 = 23$	$23 - 5 = 18$

Average Times:

- **Average TAT** = $(8 + 11 + 19 + 23) / 4 = 15.25$
- **Average WT** = $(0 + 7 + 10 + 18) / 4 = 8.75$

CPU Scheduling – Example : SJF

2) Shortest Job First (SJF - Non-Preemptive):

- Pick the process with the **shortest burst time among the available processes** in the ready queue.
- Once a process starts, it **runs till completion (non-preemptive)**.

Step 1: At time = 0:

- Only **P1** has arrived → Schedule **P1**

At time = 8:

- Now **P2 (BT=4)**, **P3 (BT=9)**, **P4 (BT=5)** are available
- Shortest BT = **P2** → Schedule **P2**

Process Management

CPU Scheduling – Example : SJF

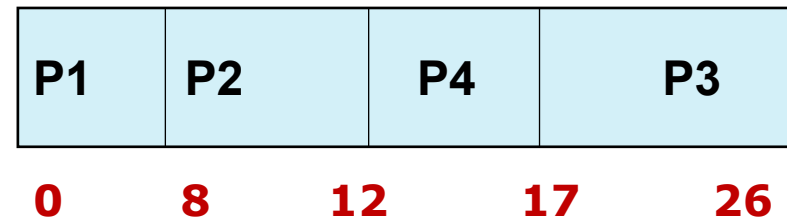
At time = 12:

- **P3 (BT=9), P4 (BT=5)** are available
- Shortest BT = **P4** → Schedule **P4**

At time = 17:

- Only **P3** is left → Schedule **P3**

Step 2: Gantt Chart



CPU Scheduling – Example : SJF

Step 3: CT, TAT, WT Calculations :

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	8	8	8 - 0 = 8	8 - 8 = 0
P2	1	4	12	12 - 1 = 11	11 - 4 = 7
P3	2	9	26	26 - 2 = 24	24 - 9 = 15
P4	3	5	17	17 - 3 = 14	14 - 5 = 9

Average Times:

Average TAT = $(8 + 11 + 24 + 14) / 4 = 14.25$

Average WT = $(0 + 7 + 15 + 9) / 4 = 7.75$

CPU Scheduling – Example : Priority

3) Priority Scheduling (Non-Preemptive):

- Among available processes, pick the one with **highest priority** (lowest number).
- **Non-preemptive:** Once started, process runs till completion.

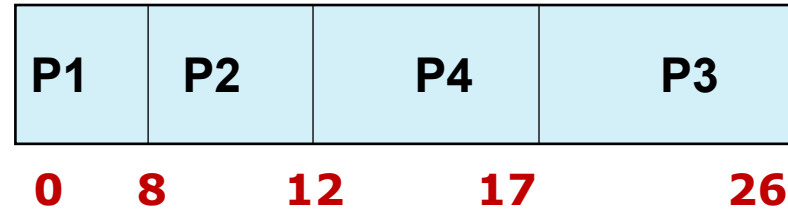
Step 1 :

- **Time 0** → Only P1 has arrived → Run P1
- **Time 8** → P2, P3, and P4 are in queue
3. Among these, **P2 has highest priority (1)** → **Run P2**
- **Time 12** → Left: P3 and P4
P4 has higher priority (2) than P3 (3) → **Run P4**
- **Time 17** → Only P3 remains → Run P3

Process Management

CPU Scheduling – Example : Priority

Step 2: Gantt Chart



Step 3: CT, TAT, WT Calculations :

Process	AT	BT	Priority	CT	TAT = CT - AT	WT = TAT - BT
P1	0	8	2	8	$8 - 0 = 8$	$8 - 8 = 0$
P2	1	4	1	12	$12 - 1 = 11$	$11 - 4 = 7$
P4	3	5	2	17	$17 - 3 = 14$	$14 - 5 = 9$
P3	2	9	3	26	$26 - 2 = 24$	$24 - 9 = 15$

Average Time:

Average TAT = $(8 + 11 + 14 + 24) / 4 = 14.25$ **Average WT** = $(0 + 7 + 9 + 15) / 4 = 7.75$

CPU Scheduling – Example : Round Robin

4) Round Robin (Preemptive):

- Each process gets **fixed time quantum in turn**.
- **Preemptive**: CPU is taken after time expires.

Let's now solve the Round Robin (RR) Scheduling Algorithm using the **same set of processes** and a **Time Quantum (TQ) = 4 units**.

- Use a queue and assign **each process a maximum of TQ** when it's their turn.
- If process **finishes** within **TQ** → **Done**.
- **Else** → Remaining **burst is put at end of queue**.

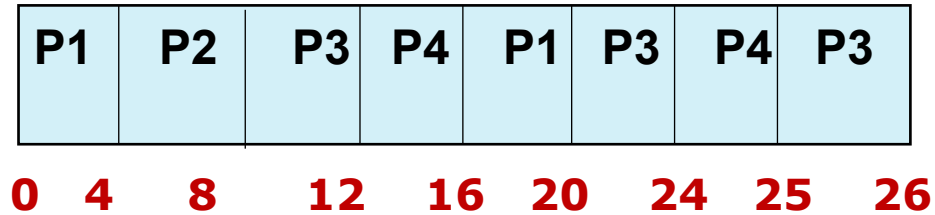
CPU Scheduling – Example : Round Robin

Step 1: Initial Queue: [P1, P2, P3, P4]

- 0-4 : P1 (runs 4 units, remaining = 4)
- 4-8 : P2 (runs 4 units, finishes)
- 8-12 : P3 (runs 4 units, remaining = 5)
- 12-16 : P4 (runs 4 units, remaining = 1)
- 16-20 : P1 (runs 4 units, finishes)
- 20-24 : P3 (runs 4 units, remaining = 1)
- 24-25 : P4 (runs 1 unit, finishes)
- 25-26 : P3 (runs 1 unit, finishes)

CPU Scheduling – Example : Round Robin

Step 2: Gantt Chart



Step 3: CT, TAT, WT Calculations

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	8	20	20 - 0 = 20	20 - 8 = 12
P2	1	4	8	8 - 1 = 7	7 - 4 = 3
P3	2	9	26	26 - 2 = 24	24 - 9 = 15
P4	3	5	25	25 - 3 = 22	22 - 5 = 17

Average Time:

Average TAT = $(20 + 7 + 24 + 22) / 4 = 18.25$ **Average WT** = $(12 + 3 + 15 + 17) / 4 = 11.75$

CPU Scheduling – Starvation

- Starvation in CPU scheduling happens when **one or more processes wait indefinitely** in the ready queue because other processes are continuously given preference.
- Scheduling Algorithms Where Starvation is Possible :**

Algorithm	Starvation Possibility	Why?
SJF (Shortest Job First)	✓ High	Long processes may never get CPU if short ones keep coming.
Priority Scheduling	✓ High	Low-priority processes may be skipped indefinitely.
FCFS (First Come First Serve)	✗ No	Once a process arrives, it will eventually be served in order.
Round Robin (RR)	✗ No	Every process gets a fair time slice , regardless of burst or priority.

Inter Process Communication

- IPC (Inter-Process Communication) is the mechanism that allows **processes to communicate** and **coordinate with each other** to share data, signals, or resources.
- In modern operating systems, multiple **processes often need to work together**.
- Since processes are isolated and have their own memory space, IPC **bridges this gap**.

Why is Inter-Process Communication (IPC) Needed?

- **To share data between processes**
 - Enables communication between independent processes to exchange information.
- **To achieve modular programming**
 - Allows different components of a system to run as separate processes and coordinate.

Inter Process Communication

- **To increase efficiency and reduce redundancy**
→ One process can use the output of another instead of repeating the same task.
- **To coordinate access to shared resources**
→ Prevents conflicts when multiple processes access files, memory, or devices.
- **To ensure proper synchronization**
→ Maintains correct execution order when processes depend on each other.
- **To enable distributed processing**
→ Useful in client-server models or multi-machine environments where processes run on different systems.

Race Condition

- A race condition is a situation where the outcome of a program depends on the timing or order of execution of threads/processes.
- It occurs when multiple threads/processes access and modify shared data concurrently without proper synchronization.

Real-Life Example – Bank ATMs

- Two people (threads) access the same bank account at the same time from two ATMs.
 - Both check balance: ₹1000.
 - Each tries to withdraw ₹700.
 - Both succeed in isolation → Balance becomes ₹-400 ❌.
- **Problem:** Shared resource (balance) was accessed without coordination.

Race Condition

Why Do Race Conditions Occur?

- Shared Resources (memory, files, variables)
- Concurrent Execution (threads or processes running at the same time)
- Lack of Synchronization (no locks or proper coordination)

Process Synchronization

- It is the technique to ensure **correct and predictable execution** when multiple **processes/threads** **access shared resources**.
- Prevents **race conditions** by controlling the **access to the shared data**.

Why is Synchronization Needed?

- To **solve race conditions**.
- To **maintain consistency** of shared data.
- To **ensure mutual exclusion, orderly execution, and data integrity**.

Process Synchronization

Critical Section (CS) Problem:

- **Critical Section:** A section of code where shared data is accessed.
- **Goal:** Ensure only one process enters its critical section at a time.

Conditions to solve:

- **Mutual Exclusion** – Only one process in critical section.
- **Progress** – Decision not postponed if no process is in CS.
- **Bounded Waiting** – A process must get a turn in finite time.

Process Synchronization

Solutions to Critical Section :

- 1. Hardware-Based : Disabling Interrupts** – Prevents context switch while in CS (used in kernel mode).
- 2. Software-Based : Peterson's Algorithm** – Classic algorithm for 2-process mutual exclusion using flags and turn.

3. Synchronization Tools:

- Semaphores – Signaling mechanism (wait/signal).
- Mutex (Mutual Exclusion Object) – Binary lock used in OS & threading.
- Monitors – High-level abstraction that wraps shared variables and methods.

Process Synchronization

3.1. Semaphores

- **What:** A semaphore is a signaling mechanism used to control access to a shared resource by multiple processes.
- **How:** It uses two atomic operations — wait() (also called P) and signal() (also called V).
- **Where:** Commonly used in OS-level resource management (e.g., printers, memory slots).

Example (Real Life):

- Imagine a parking lot with 3 spots. Only 3 cars can park at a time. If the lot is full, new cars (processes) must wait.

Process Synchronization

3.2. Mutex (Mutual Exclusion)

- **What:** A mutex is a locking mechanism that allows only one thread/process to access a critical section at a time.
- **How:** It acts like a binary flag (locked/unlocked).
- **Where:** Widely used in threaded applications to prevent race conditions.

Example (Real Life):

- A single-user bathroom — one person uses it at a time, and others wait outside with the key (mutex).

Process Synchronization

3.3 Monitor

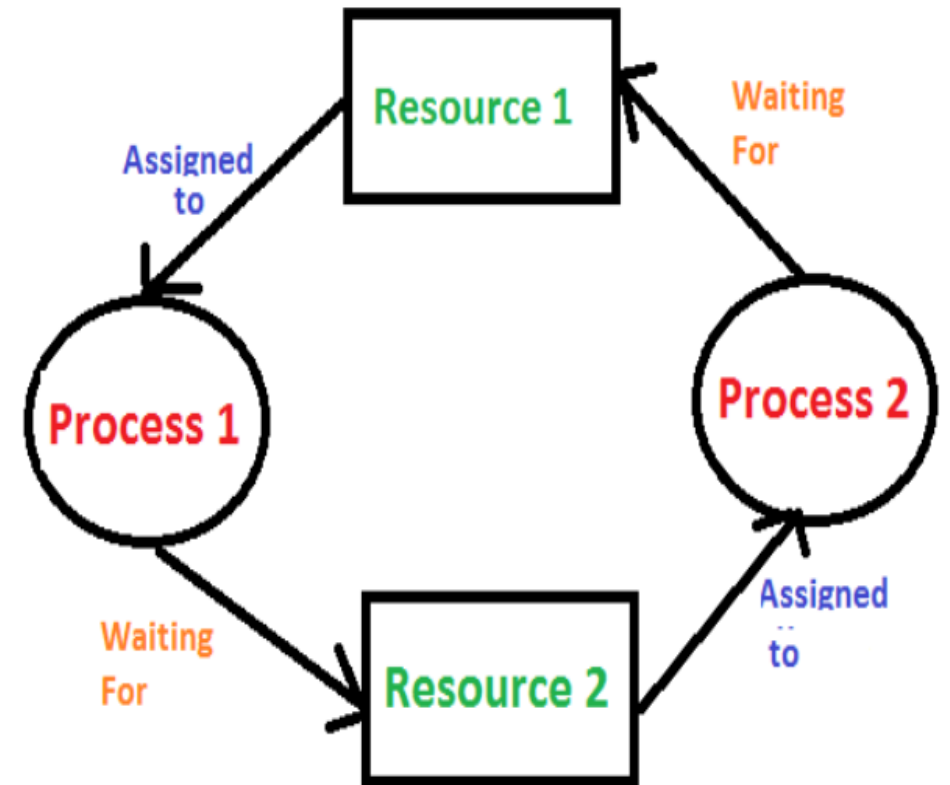
- **What:** A monitor is a **high-level abstraction** that wraps shared data and synchronization code into a single construct.
- **How:** It ensures **only one process** executes in the monitor at a time.
- **Where:** Built-in in some programming languages like Java (synchronized block).

Example (Real Life):

Think of a **hotel front desk**: guests queue up, and the receptionist (monitor) handles one guest at a time.

Deadlock

- A **deadlock** is a situation where a group of processes are **waiting for each other indefinitely**, and **none of them can proceed**.
- **Example** : Process P1 is holding Resource R1 and waiting for Resource R2.
- Meanwhile, Process P2 is holding Resource R2 and waiting for Resource R1.
- As a result, both processes are waiting for each other indefinitely - **This is a classic Deadlock scenario.**



Deadlock Handling

- Deadlock occurs when processes wait indefinitely for resources.
- OS handles deadlocks in three ways:
 - 1) **Prevention**
 - 2) **Avoidance**
 - 3) **Detection & Recovery**

Deadlock Handling

1) Deadlock Prevention

- Prevent deadlocks by **ensuring that at least one of the Coffman conditions never holds.**
- **Deadlock Conditions (Coffman's Conditions)**
 - **Mutual Exclusion:** Make resources sharable (not always possible).
 - **Hold and Wait:** Require a process to request *all* resources at once.
 - **No Preemption:** Allow OS to forcefully preempt resources.
 - **Circular Wait:** Impose a strict ordering of resource allocation.
- **Example:** Force process to request printer and scanner at the same time; if not available, request is denied **and retried later.**

Deadlock Handling

2) Deadlock Avoidance

- Allow deadlocks **only if the system can guarantee a safe state.**

Key Tool:

- **Banker's Algorithm:** Grants resources only if it **leaves the system in a safe state.**
- **Example:** Like a bank ensuring that giving a loan won't **prevent other customers** from withdrawing their money.

Deadlock Handling

3) Deadlock Detection and Recovery

- Let deadlocks happen but **detect and recover from them**.

This is done using :

- A **Resource Allocation Graph (RAG)** or **deadlock detection algorithm** to identify deadlocks.
- Recovery options:
 - Terminate one or more processes.
 - Preempt resources and restart processes.

Example: OS checks periodically; if it finds a cycle (deadlock), it kills the least important process.

Memory Management

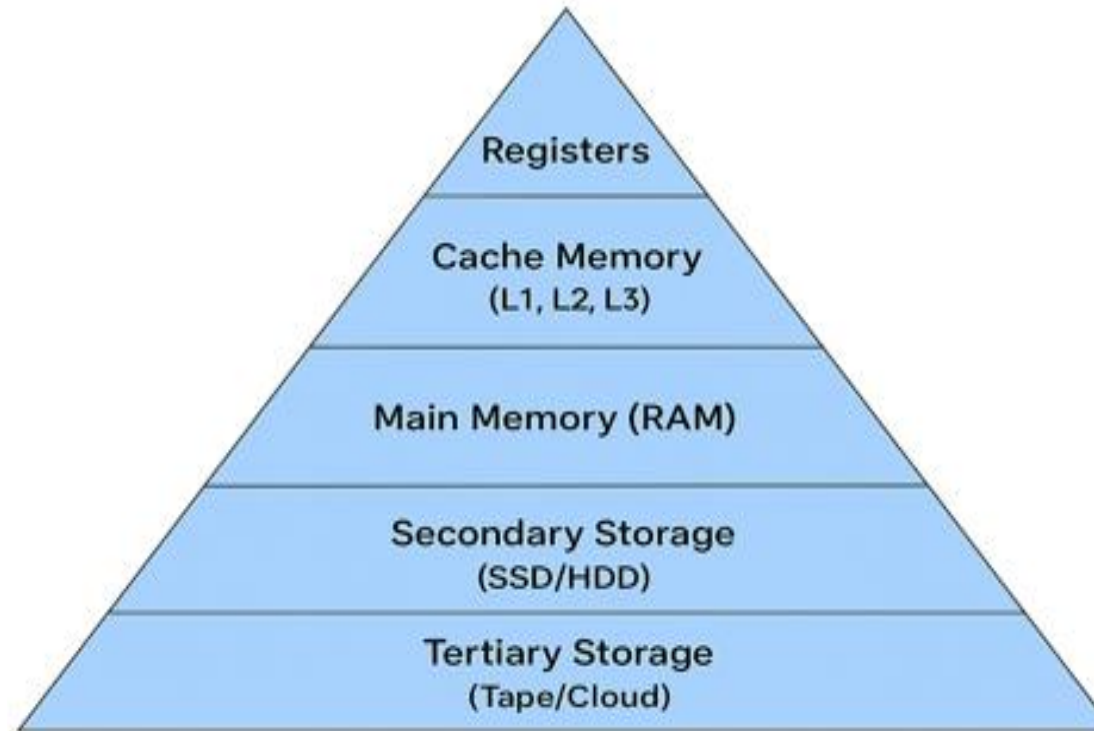


Introduction

- Memory management is a **core function of the Operating System** that handles the **allocation** and **deallocation** of **memory to processes**.
- **Responsibilities of Memory Management :**
 - Tracking memory usage
 - Allocating memory to processes
 - Deallocating memory when no longer needed
 - Managing virtual memory

Memory Hierarchy in Operating Systems

- The Memory Hierarchy is a structure that organizes memory components based on **speed, cost, and capacity**.



Memory Hierarchy

Memory Hierarchy in Operating Systems

Registers :

- These are the smallest and fastest memory units located **inside the CPU**.
- Used to **hold data and instructions** that the CPU is currently processing.
- Think of them as your brain's immediate thoughts.

Cache Memory :

- A very fast memory located **close to the CPU**, used to store **frequently accessed data** from RAM.
- Helps reduce the time taken by the **CPU to fetch data**.
- Like sticky notes you keep on your desk for quick reference.

Memory Hierarchy in Operating Systems

Main Memory (RAM) :

- Also called **primary memory**, RAM holds all the **programs and data** that are actively used by the system.
- It is **faster than secondary memory**, but slower than cache.
- Like a notebook open on your desk.

Secondary Memory:

- This includes **Hard Disks or SSDs**, where all **programs and files** are stored permanently.
- **Slower than RAM**, but has much larger storage capacity.
- Like a drawer where you store all your documents.

Memory Hierarchy in Operating Systems

Tertiary Memory :

- This refers to **external storage devices** such as CDs, DVDs, and backup tapes.
- Used for **long-term backup** and data archiving.
- Like files stored in your storeroom that you access rarely.

Memory Hierarchy : Summary

Type	Speed	Cost	Capacity	Volatile
Registers	Fastest	Very High	Very Low	Yes
Cache Memory (L1, L2, L3)	Very Fast	High	Low	Yes
Main Memory (RAM)	Fast	Moderate	Medium	Yes
Secondary Storage (SSD/HDD)	Slower	Low	High	No
Tertiary Storage (Tape/Cloud)	Slowest	Very Low	Very High	No

Logical vs Physical Address

- **Logical Address** is the address a program uses to **access memory**. It's generated by the CPU during program execution.
- **Physical Address** is the **actual location** in the main memory (RAM) where **data is stored**.
- The **Memory Management Unit (MMU)** in the OS is responsible for **translating a logical address into a physical address**.
- The user or program **only works** with **logical addresses**. It **doesn't know** where the data is stored physically.

Fragmentation

- **Fragmentation** refers to **wasted memory space** that occurs when memory is allocated and deallocated over time.
- It is broadly classified into two types:

1) Internal Fragmentation

2) External Fragmentation

Fragmentation

1) Internal Fragmentation :

- Occurs when allocated memory is **larger than** the memory required by the process.
- The **unused space within the allocated block** is wasted.
- Common in **Fixed Partitioning** systems.

Example :

- A fixed partition is 8 KB.
- A process needs only 6 KB.
- **2 KB is wasted** internally.

Fragmentation

2) External Fragmentation :

- Occurs when **free memory is scattered** throughout the system.
- Total free memory is enough to satisfy a request, but **not contiguous**.
- Common in **Variable Partitioning** systems.

Example:

- Three free blocks of 2 KB, 4 KB, and 6 KB exist.
- A process needs 8 KB.
- Although 12 KB is free in total, there's **no single 8 KB block**.

Memory Management Techniques

- Memory management in operating systems refers to the **way the system handles primary memory**.
- It is broadly classified into:

1. Contiguous Memory Allocation

2. Non - Contiguous Memory Allocation

Contiguous Memory Allocation

1. Contiguous Memory Allocation

- Entire process is allocated memory in a **single contiguous block**.
- Simpler to implement but can lead to **external fragmentation**.
- Techniques:
 - **Single Partition Allocation**
 - **Multiple Partition Allocation**
 1. Fixed-sized partitions
 2. Variable-sized partitions
 - **Allocation Strategies**
 1. First Fit
 2. Best Fit
 3. Worst Fit

Contiguous Memory Allocation : Memory Allocation Strategies

1. First Fit :

- Allocates the **first available** memory block that is **large enough** for the process.
- **Fast and simple** — scans memory from the beginning.

Example:

- Free blocks: 4 KB, 8 KB, 6 KB
- Request: 5 K
- Allocated to the **8 KB block (first fit found)**.

Pros: Fast allocation

Cons: May lead to external fragmentation

Contiguous Memory Allocation : Memory Allocation Strategies

2. Best Fit :

- Searches for the **smallest free block** that is **big enough** for the process.
- Tries to minimize **leftover space** (internal fragmentation).

Example:

- Free blocks: 4 KB, 8 KB, 6 KB
- Request: 5 KB
- Allocated to the **6 KB block (best fit found)**

Pros: Minimizes wasted memory

Cons: **Slower** than First Fit (needs full scan) , Can still lead to **external fragmentation**

Contiguous Memory Allocation : Memory Allocation Strategies

3. Worst Fit :

- Allocates memory to the **largest available block**.
- Idea: leave **larger leftover spaces** that might be useful later.

Example:

- Free blocks: 4 KB, 8 KB, 6 KB
- Request: 5 KB
- Allocated to the **8 KB block (worst fit)**

Pros: Leaves big chunks free

Cons: Often inefficient , May increase fragmentation

Non - Contiguous Memory Allocation

- Process is divided and stored in **different memory blocks**, not necessarily adjacent.
- Solves **external fragmentation problem**.

Techniques:

1. Paging

- Divides physical memory and process memory into **equal-sized blocks** (pages & frames).
- No external fragmentation, but **internal fragmentation possible**.

2. Segmentation

- Divides memory logically based on **code, data, stack**, etc.
- More natural view for programmers, but **external fragmentation possible**.

3. Paging + Segmentation

- Combines benefits of both techniques.

Non - Contiguous Memory Allocation : Paging

- Paging is a **non-contiguous memory allocation technique** that divides both **physical memory** and **logical memory** into **fixed-size blocks**.
 - **Logical memory** → divided into **pages**
 - **Physical memory** → divided into **frames**
- The OS maps pages to available frames using a **page table**.

Non - Contiguous Memory Allocation : Paging

- The Page Table is a data structure used by the Operating System to **map logical pages (used by a program) to physical frames (actual locations in RAM).**
- **Why Paging is required?**
 - Solves the **external fragmentation problem**
 - Efficient memory **usage**
 - Simplifies memory **allocation**

Non - Contiguous Memory Allocation : Paging

Steps Involved in Paging :

Step 1: Divide Memory

- Logical Memory (used by program) is divided into Pages (equal-sized blocks).
- Physical Memory (RAM) is divided into Frames (same size as pages).
- *Think of pages and frames like same-sized boxes — pages are what the program wants, and frames are what RAM offers.*

Step 2: Allocate Pages

- The operating system loads each page of the process into any available frame in physical memory.
- *No need for continuous space in RAM — pages can go into any free frame.*

Non - Contiguous Memory Allocation : Paging

Step 3: Create Page Table

- A Page Table is created to keep track of where each page is stored.
 - It maps each Page Number to a Frame Number.
- *This is like a list that tells: "Page 0 is in Frame 5", "Page 1 is in Frame 2", etc.*

Step 4: Translate Address

- When the CPU generates a logical address (Page Number + Offset), the MMU (Memory Management Unit) uses the Page Table to find the corresponding Frame.
 - The physical address is calculated as: $\text{Frame Number} * \text{Frame Size} + \text{Offset}$.
- *It's like using a postal code (Page Number) and house number (Offset) to find the exact location in memory.*

Non - Contiguous Memory Allocation : Paging

Step 5: Handle Page Fault

- If the required page is not in memory, it's a Page Fault.
- The OS brings the missing page from disk (secondary memory) into a free frame, updates the Page Table, and resumes execution.
- *This is like realizing your document is in Google Drive (disk), so you download it (load into RAM) before you can read it.*

Step 6: Run Program

- During program execution, every memory access uses the Page Table to translate logical addresses to physical ones.
- *The CPU just asks, "I want Page 3, Offset 2" — the MMU and Page Table do the rest!*

Non - Contiguous Memory Allocation : Paging

Example Logical Address: 2050

- We will translate this **logical address** into a **physical address**.

Given:

- **Page size** = 1 KB (1024 bytes)
- **Logical Address Space** = 4 pages → Page 0 to Page 3
- **Physical Memory** = 8 frames → Frame 0 to Frame 7

Non - Contiguous Memory Allocation : Paging

1. Divide the Logical Address :

- We break the logical address 2050 into two parts:
- **Page Number = logical Address / Page Size**
- Page Number = $2050 \div 1024 = 2$ (quotient)
- **Offset = Logical Address mod Page Size**
- Offset = $2050 \% 1024 = 2$ (remainder)

2. Use the page table:

From the page table

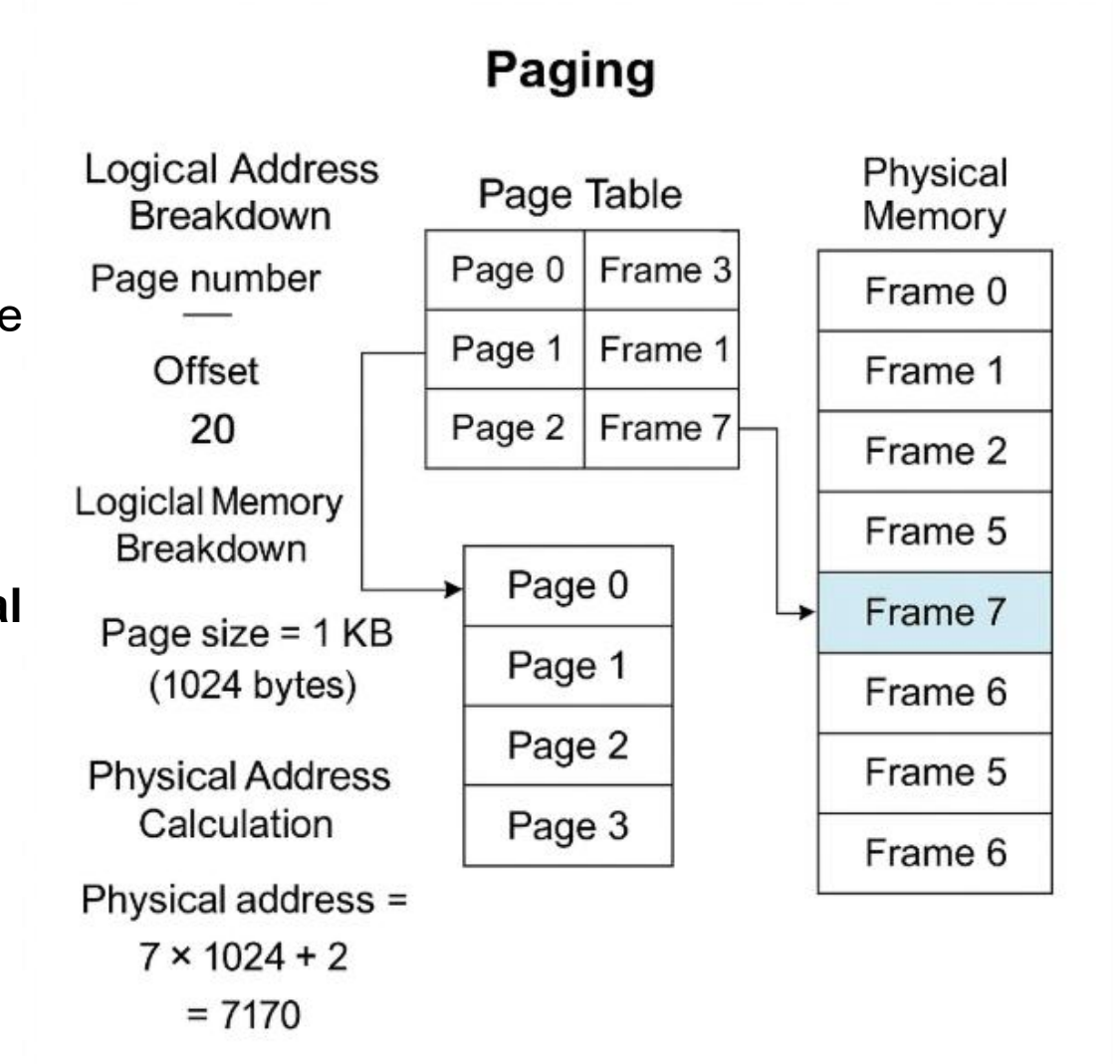
Page No	Frame No
0	3
1	1
2	7

- So, **Page 2** is currently stored in **Frame 7** of physical memory.

Non - Contiguous Memory Allocation : Paging

3. Calculate Physical Address :

- **Frame size** = 1024 bytes
- **Physical Address** = (Frame Number × Frame Size) + Offset
- **Physical Address** = $(7 \times 1024) + 2 = 7170$
- **Logical Address 2050 is mapped to Physical Address 7170**



Non - Contiguous Memory Allocation : Paging

Advantages

- No external fragmentation
- Easy to manage memory
- Fast memory access with hardware support (TLB)

Disadvantages

- Internal fragmentation (last page may not be fully used)
- Page table overhead

Non - Contiguous Memory Allocation : Segmentation

What is Segmentation?

- Segmentation is a **non-contiguous memory allocation** technique where memory is divided based on **logical program segments**, not fixed sizes.
- Each program is divided into logical segments:
 - **Code segment**
 - **Data segment**
 - **Stack segment**
 - **Heap segment**, etc.
- Each segment has:
 - **A Base Address** (where it starts in physical memory)
 - **A Limit** (length/size of the segment)

Non - Contiguous Memory Allocation : Segmentation

Why Use Segmentation?

- Logical separation of program components
- Enables **modular programming**
- Allows **independent** protection of segments
- Avoids **internal fragmentation** (since size varies)

Non - Contiguous Memory Allocation : Segmentation

Steps Involved in Segmentation :

Step 1: Divide Memory Logically :

- The compiler breaks the program into logical segments like:
 - Segment 0: Code
 - Segment 1: Data
 - Segment 2: Stack
- *These segments represent meaningful units in the program.*

Step 2: Allocate Segments in RAM :

- The OS loads each segment into **available memory blocks** (non-contiguous).
- Segments can be of **different sizes**.
- *Segments are not equal-sized like paging — they match the component size.*

Non - Contiguous Memory Allocation : Segmentation

Step 3: Create Segment Table :

- The **OS creates** a Segment Table.
- Each entry contains:
 - **Base Address** → starting point in physical memory
 - **Limit** → segment size

➤ *It's like saying "Code starts at 400, size 1000".*

Step 4: Translate Logical Address :

- The CPU generates a **logical address in the form** : Segment Number + Offset
 - The MMU uses the **Segment Table**:
 - Checks if Offset < Limit
 - Then computes: **Physical Address = Base + Offset**
- *Like asking: "Go to the stack, 20 bytes in" → Stack starts at 5000 → Final address = 5020*

Non - Contiguous Memory Allocation : Segmentation

Step 5: Handle Segment Fault :

- If the **Offset > Limit** → **Segment Fault**
- OS **halts** the process or raises a protection error.
- *Prevents a process from accessing memory outside its boundary.*

Step 6: Run Program :

- Each memory reference is translated using the **Segment Table**.
- Provides **protection, separation, and flexibility**.
- *CPU simply asks: “Segment 1, Offset 25” → MMU finds it and maps to the physical address.*

Non - Contiguous Memory Allocation : Segmentation

Example :

Given a Segment Table:

Segment No	Base	Limit
0 (Code)	1000	500
1 (Data)	2000	400
2 (Stack)	5000	300

- **Logical Address** = Segment 1, Offset = 120
- We need to translate a **Logical Address to a Physical Address Using a Segment Table.**

Non - Contiguous Memory Allocation : Segmentation

Step-by-Step:

1. Check Limit:

- Offset (120) < Limit (400) → Valid

2. Translate Address:

- Physical Address = Base + Offset = 2000 + 120 = 2120
- **Logical Address (Segment 1, 120) → Physical Address 2120**

Non - Contiguous Memory Allocation : Segmentation

Segmentation

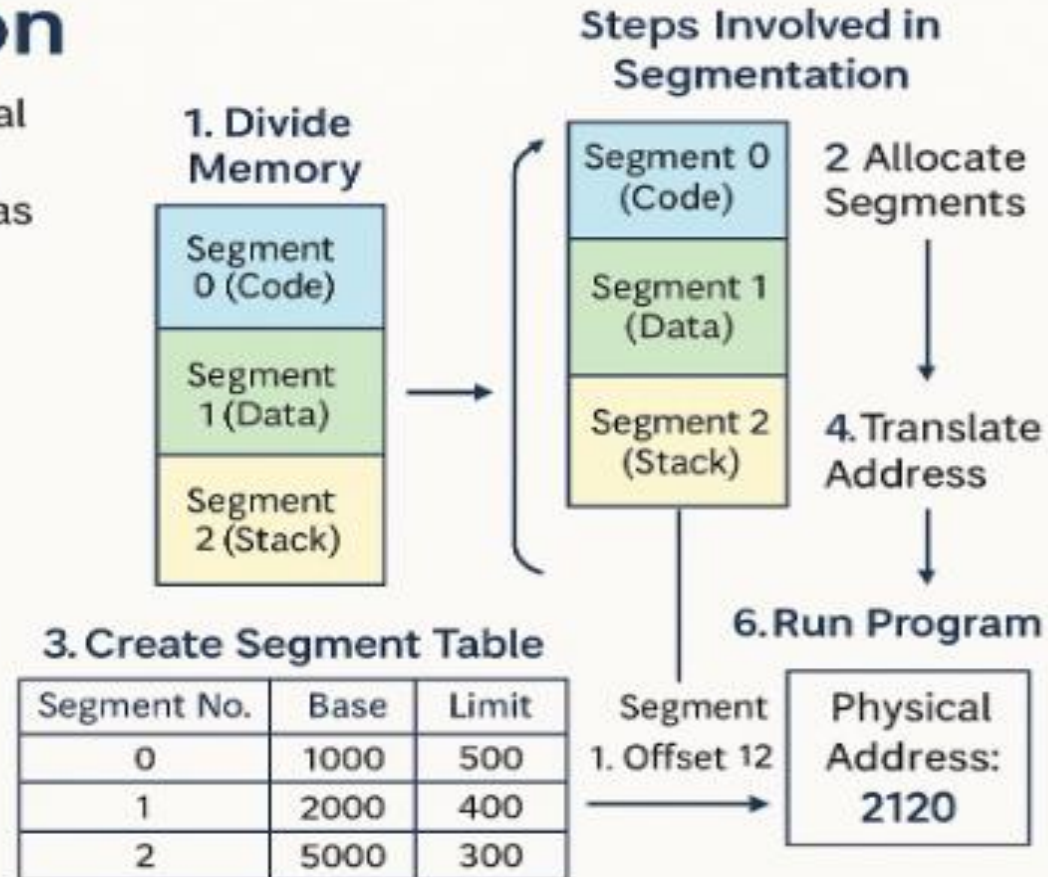
Segmentation divides logical memory into variable-size segments. Each segment has a base address and a limit

Why Segmentation?

- Logical separation of program components
- Avoids internal fragmentation
- Supports modularity and protection

Steps Involved in Segmentation

- Logical memory
- Avoids internal alias



Non - Contiguous Memory Allocation : Segmentation

Advantages

- Logical program structure → **better modularity**
- Allows **independent segment protection**
- Avoids **internal fragmentation**

Disadvantages

- Can suffer from **external fragmentation**
- Requires **Segment Table**
- More **complex** than **paging** in address translation

Paging vs Segmentation

Paging	Segmentation
Memory is divided into fixed-size blocks called pages.	Memory is divided into variable-size blocks called segments.
It is based on the physical division of memory.	It is based on the logical division of a program.
All pages are of equal size.	Segments can be of different sizes like code, stack, data.
Causes internal fragmentation.	Causes external fragmentation.
The user does not see the division.	The division is visible to the user/programmer.
Address is split into page number and offset.	Address is split into segment number and offset.
Efficient in memory management due to fixed-size blocks.	More flexible for program structure and protection.

Swapping

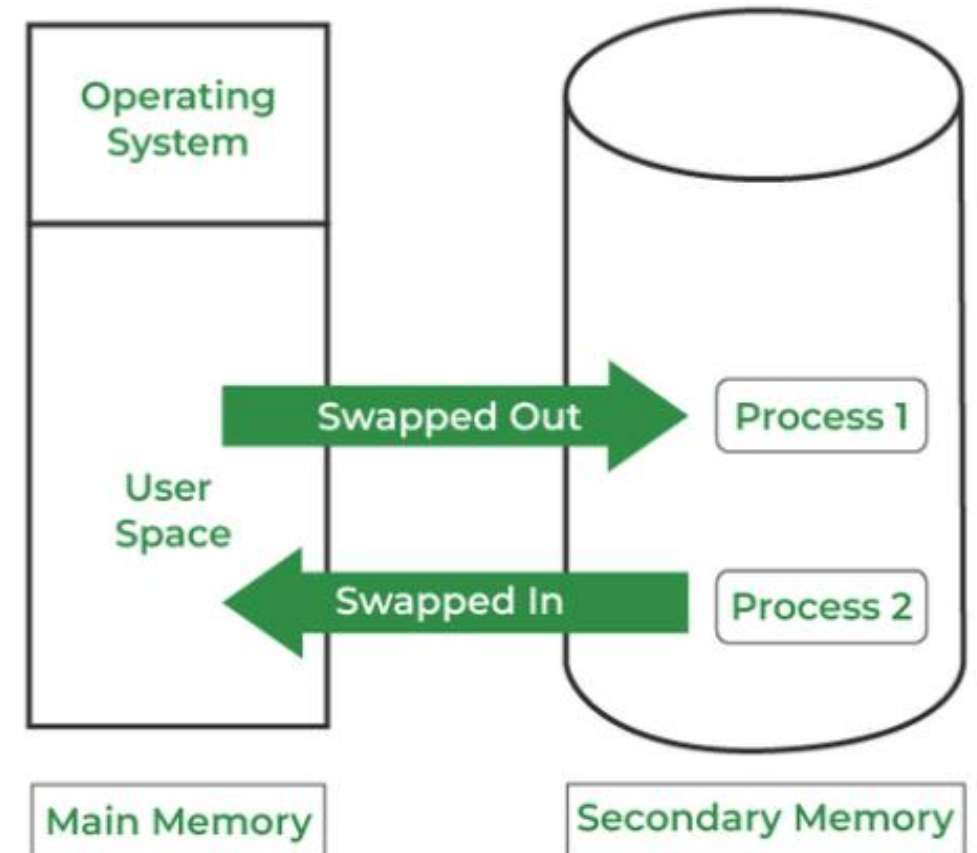
- Swapping is a memory management technique where a process is temporarily moved **from main memory (RAM) to secondary storage (disk)** to free up RAM for other processes.
- Later, the process can be **swapped back into RAM when needed**.
- This allows the system to handle **more processes** than the available physical memory can hold at once, **enabling multiprogramming**.
- However, **frequent swapping** can **slow down performance**, a situation known as **thrashing**.

Memory Management

Swapping

Swapping has been subdivided into two concepts:

- **Swap-out** is a technique for **moving a process from RAM to the hard disc.**
- **Swap-in** is a method of transferring a program from a **hard disc to main memory, or RAM.**



Virtual Memory & Demand Paging

Virtual Memory :

- A technique that enables processes to use **more memory than physically available**
- **Part of the process** is in memory; the rest is on disk.
- Gives **illusion of large memory** space.

Demand Paging :

- Pages are loaded into RAM **only when needed** (lazy loading).
- Reduces memory use, but can lead to **page faults**.
- Uses **page tables and disk swapping**.

File System Management



File System

- A file system is a method an operating system uses to **store, organize, and manage files and directories on a storage device.**
- Some common types of file systems include:
 - **FAT (File Allocation Table)** : An older file system used by older versions of Windows and other operating systems.
 - **NTFS (New Technology File System)**: A modern file system used by Windows. It supports features such as file and folder permissions, compression, and encryption.
 - **ext (Extended File System)**: A file system commonly used on Linux and Unix-based operating systems.
 - **HFS (Hierarchical File System)**: A file system used by macOS.
 - **APFS (Apple File System)**: A new file system introduced by Apple for their Macs and iOS devices.

File Directories

- The **collection of files** is a file directory. The directory contains **information about the files**, including **attributes, location, and ownership**.
- Below are **information** contained in a device directory.
 - Name, Type, Address, Current length, Maximum length
 - Date last accessed, Date last updated, Owner id, Protection information
- The **operation** performed on the directory are:
 - Search for a file, Create a file, Delete a file
 - List a directory, Rename a file, Traverse the file system

File Allocation Methods

- The allocation methods define **how the files are stored** in the disk blocks.
- There are three main disk space or file **allocation methods**.
 - **Contiguous Allocation**
 - **Linked Allocation**
 - **Indexed Allocation**
- The main idea behind these methods is to provide:
 - Efficient **disk space utilization**.
 - **Fast access** to the file blocks.

File Allocation Methods

1. Contiguous Allocation

- **How it works:** All the blocks of a file are stored **together in adjacent locations** on the disk.
- **Use Case:** Suitable for files with a **known fixed size**, like media files.
- **Advantages:**
 - Very **fast access**, especially for **sequential reads**.
 - **Simple** to implement and manage.
- **Disadvantages:**
 - **External fragmentation** (free space gets scattered).
 - **Difficult to grow files** after allocation.
 - Requires knowing the **entire file size in advance**.

File Allocation Methods

2. Linked Allocation

- **How it works:** Each file is stored as a **chain of disk blocks**, with each block pointing to the next
- **Use Case:** Good for files that are frequently **updated or grow dynamically**.
- **Advantages:**
 - **No external** fragmentation.
 - Easy to **grow files** without relocating.
- **Disadvantages:**
 - **Slower random access** (must follow pointers).
 - Extra **space used for pointers**.
 - **Risk of broken chains** if a pointer is corrupted.

File Allocation Methods

3. Indexed Allocation

- **How it works:** An **index block** is created for each file that contains **addresses of all data blocks**.
- **Use Case:** Suitable when **both random and sequential** access are needed.
- **Advantages:**
 - **Direct access** to any block (fast random access).
 - **No external fragmentation**.
- **Disadvantages:**
 - **Overhead** of storing the index block.
 - For **large** files, may require **multi-level indexing**.

Disk Management



Introduction

- The operating system needs to track the **location of the disk** for every part of every file on the disk.
- In some cases, this means tracking **hundreds of thousands** of files and file fragments on **a single physical disk**.
- Disk management of the operating system includes:
 - **Disk Format**
 - **Booting from disk**
 - **Bad block recovery**

Disk Management Techniques

- **Partitioning:** Splits a **disk into parts** to organize and manage data better.
- **Formatting:** Prepares a **partition** for use by **creating** a file system and **erasing** old data.
- **File System Management:** Controls how data is stored and accessed using **file systems like NTFS or ext4**.
- **Disk Space Allocation:** Assigns space on the disk for files using methods like **contiguous or indexed allocation**.
- **Disk Defragmentation:** **Rearranges scattered** data to speed up file access and improve performance.

Disk Scheduling

- Disk Scheduling is a technique used by operating systems to decide the order in which **disk I/O requests** are serviced.
- Since **disk access is much slower** compared to memory, efficient disk scheduling helps improve overall system performance.
- **Why Disk Scheduling is needed ?**
 - To **minimize disk seek time** (head movement).
 - To improve the **throughput** of the system.
 - To reduce **latency** and **response time**.

Disk Scheduling

Important Terminologies:

Term	Meaning
Seek Time	Time to move the disk arm to the desired track.
Rotational Delay	Time waiting for the disk to rotate to the correct sector.
Transfer Time	Time to transfer data to/from the disk.
Disk Arm	The part that moves across the disk surface to read/write data.

Disk Scheduling

Commonly used Disk Scheduling Algorithms:

- FCFS
- SSTF
- SCAN
- LOOK
- C-SCAN
- C-LOOK
- N-Step SCAN

Disk Scheduling

Example : We will calculate the Total Head Movement for each algorithm.

Given:

- **Initial Head Position:** 53
- **Request Queue:** 98, 183, 37, 122, 14, 124, 65, 67
- **Disk Range:** 0 to 199 (assumed)
- **Disk Queue:** 98, 183, 37, 122, 14, 124, 65, 67
- **Initial Head Position:** 53

Disk Scheduling

1) FCFS (First-Come, First-Served)

- Processes requests in the **order they arrive, without reordering.**

Step-by-step Solution:

1. Start at 53.

2. Go to 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67.

Total Head Movement:

$$\begin{aligned} &|53-98| + |98-183| + |183-37| + |37-122| + |122-14| + |14-124| + |124-65| + |65-67| = 45 + 85 + 146 + 85 \\ &+ 108 + 110 + 59 + 2 = 640 \text{ cylinders} \end{aligned}$$

Gantt Chart:

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Disk Scheduling

2) SSTF (Shortest Seek Time First)

- Selects the **disk I/O request closest** to the current head position to **minimize seek time**.

Step-by-step Solution:

- **Initial Head Position: 53**
Request Queue: 98, 183, 37, 122, 14, 124, 65, 67

Step	Current Head	Remaining Requests	Nearest Request	Movement
1	53	98, 183, 37, 122, 14, 124, 65, 67	65	12
2	65	98, 183, 37, 122, 14, 124, 67	67	2
3	67	98, 183, 37, 122, 14, 124	98	31
4	98	183, 37, 122, 14, 124	122	24
5	122	183, 37, 14, 124	124	2
6	124	183, 37, 14	183	59
7	183	37, 14	37	146
8	37	14	14	23

Disk Management

Disk Scheduling

Total Head Movement:

$$12 + 2 + 31 + 24 + 2 + 59 + 146 + 23 = \mathbf{299 \text{ cylinders}}$$

Gantt Chart:

53 → 65 → 67 → 98 → 122 → 124 → 183 → 37 → 14

Note: SSTF reduces head movement compared to FCFS by selecting closest jobs first.

Disk Scheduling

3) SCAN (Elevator Algorithm)

- The disk arm moves in **one direction (like an elevator)**, servicing all requests until it reaches the end, then **reverses direction**.

Step-by-step (Moving Left First):

1. Sort Requests:

[14, 37, 65, 67, 98, 122, 124, 183]

2. Split Based on Direction:

- Left of 53: [14, 37]
- Right of 53: [65, 67, 98, 122, 124, 183]

3. Service Order (SCAN left to 0, then reverse):

53 → 37 → 14 → 0 (end) → 65 → 67 → 98 → 122 → 124 → 183

Gantt Chart:

53 → 37 → 14 → 0 → 65 → 67 → 98 → 122 → 124 → 183

Disk Management

Disk Scheduling

Head Movements:

- $53 \rightarrow 37 = 16$
- $37 \rightarrow 14 = 23$
- $14 \rightarrow 0 = 14$
- $0 \rightarrow 65 = 65$
- $65 \rightarrow 67 = 2$
- $67 \rightarrow 98 = 31$
- $98 \rightarrow 122 = 24$
- $122 \rightarrow 124 = 2$
- $124 \rightarrow 183 = 59$

Total Head Movement = $16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 = 236$ cylinders

- **SCAN** improves upon SSTF by **preventing starvation and servicing** requests in a fair sequence.

Disk Scheduling

4) LOOK Algorithm

LOOK is similar to SCAN, but instead of going to the **end (like track 0 or max)**, the disk arm only goes as far as the **last request** in the current **direction and then reverses**.

Step-by-Step (Moving Left First):

1. Sorted Queue:

[14, 37, 65, 67, 98, 122, 124, 183]

2. Split Based on Direction:

- Left of 53: [14, 37]
- Right of 53: [65, 67, 98, 122, 124, 183]

Disk Scheduling

3. Service Order (LOOK left to furthest left request, then reverse):

53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

(Notice: Unlike SCAN, we don't go to 0, we stop at 14 and reverse)

Gantt Chart:

53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

Disk Management

Disk Scheduling

Head Movements:

- $53 \rightarrow 37 = 16$
- $37 \rightarrow 14 = 23$
- $14 \rightarrow 65 = 51$
- $65 \rightarrow 67 = 2$
- $67 \rightarrow 98 = 31$
- $98 \rightarrow 122 = 24$
- $122 \rightarrow 124 = 2$
- $124 \rightarrow 183 = 59$

Total Head Movement = $16 + 23 + 51 + 2 + 31 + 24 + 2 + 59 = 208$ cylinders

Note: *LOOK is more efficient than SCAN as it avoids unnecessary movement to the end of the disk.*

Disk Scheduling

5) C-SCAN (Circular SCAN)

- In C-SCAN, the disk arm moves in one direction (e.g., toward 0), services all requests until it reaches the end, and then **jumps to the other end** of the disk and starts servicing requests in the **same direction again**. This provides **more uniform wait time** compared to SCAN.

Step-by-Step:

1. Sorted Request Queue:

[14, 37, 65, 67, 98, 122, 124, 183]

2. Split Requests:

- Left of 53: [14, 37]
- Right of 53: [65, 67, 98, 122, 124, 183]

Disk Scheduling

3. Service Order (C-SCAN, Left then Jump to Max End):

53 → 37 → 14 → 0 → 199 → 183 → 124 → 122 → 98 → 67 → 65

(Head moves to 0 even if there are no requests at 0, jumps to 199, and continues)

Gantt Chart:

53 → 37 → 14 → 0 → 199 → 183 → 124 → 122 → 98 → 67 → 65

Head Movements:

- 53 → 37 = 16
- 37 → 14 = 23
- 14 → 0 = 14
- 67 → 65 = 2

Disk Management

Disk Scheduling

- $0 \rightarrow 199$ (jump) = 199
- $199 \rightarrow 183 = 16$
- $183 \rightarrow 124 = 59$
- $124 \rightarrow 122 = 2$
- $122 \rightarrow 98 = 24$
- $98 \rightarrow 67 = 31$

Total Head Movement = $16 + 23 + 14 + 199 + 16 + 59 + 2 + 24 + 31 + 2 = 386$ cylinders

Note: C-SCAN provides uniform wait time but has higher total head movement due to the circular jump.

Disk Scheduling

6) C-LOOK (Circular LOOK)

- C-LOOK is a **variant of C-SCAN**. Instead of going all the way to the end of the disk, the **head only goes** as far as the **last request** in that direction, **then jumps to the farthest request** on the other end and continues in the same direction. This reduces **unnecessary head movement**.

Step-by-Step:

1. Sorted Request Queue:

[14, 37, 65, 67, 98, 122, 124, 183]

2. Split Requests:

- Left of 53: [14, 37]
- Right of 53: [65, 67, 98, 122, 124, 183]

Disk Scheduling

3. Service Order (C-LOOK):

53 → 37 → 14 → 183 → 124 → 122 → 98 → 67 → 65

(After servicing 37 and 14, jump to highest request (183) and move backward in decreasing order)

Gantt Chart:

53 → 37 → 14 → [JUMP] → 183 → 124 → 122 → 98 → 67 → 65

Head Movements:

- 53 → 37 = 16
- 37 → 14 = 23
- 14 → 183 (jump) = 169
- 183 → 124 = 59

Disk Management

Disk Scheduling

- $124 \rightarrow 122 = 2$
- $122 \rightarrow 98 = 24$
- $98 \rightarrow 67 = 31$
- $67 \rightarrow 65 = 2$

Total Head Movement = $16 + 23 + 169 + 59 + 2 + 24 + 31 + 2 = 326$ cylinders

Note: C-LOOK is more efficient than C-SCAN since it avoids traveling to disk extremes unnecessarily while still providing fairness.

System Calls



System Calls

What is System Call?

- A System Call is a way for programs to **request services from the operating system**.
- It acts as a bridge between **user programs and the OS kernel**.
- Programs use system calls to perform **low-level operations** like reading files, creating processes, or using memory.
- **Without** system calls, user programs **cannot directly access hardware or OS resources**.
- Common examples of system calls:
 - **read()** – to read data from a file
 - **write()** – to write data to a file
 - **fork()** – to create a new process
 - **exec()** – to run a new program
 - **exit()** – to end a program

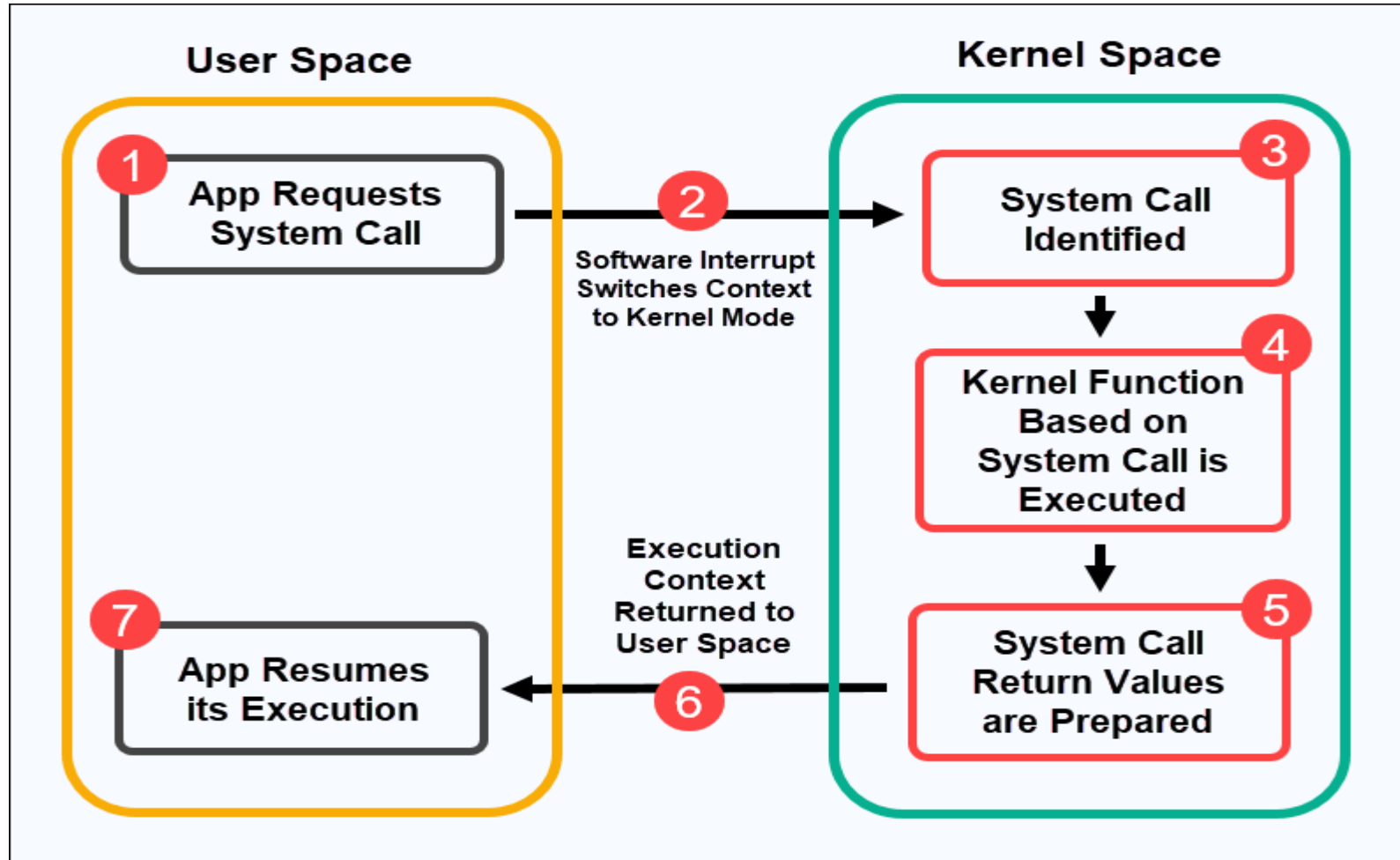
System Calls

Purpose of System Call

- **Access Hardware Resources** : Allows user programs to interact with hardware like disk, printer, and memory.
- **Program Control** : Helps in creating, executing, or terminating processes.
- **File Management** : Used for reading, writing, creating, or deleting files.
- **Communication** : Enables inter-process communication (IPC) like sending or receiving messages.
- **Security and Access Control** : Manages permissions and ensures secure access to system resources.
- **Memory Management** : Allocates and deallocates memory to processes.

System Calls

Working of System Call



System Calls

Working of System Call

Step by Step explanation of how System Call works:

- 1) **System Call Request** – Application calls a function like `read()` to request a service.
- 2) **Switch to Kernel Mode** – A software interrupt switches the CPU from user mode to kernel mode.
- 3) **Identify System Call** – OS identifies the requested system call using a unique ID.
- 4) **Execute Kernel Function** – The kernel performs the requested operation (e.g., reads file data).
- 5) **Prepare Return Values** – Results are stored and prepared to return to the application.
- 6) **Switch to User Mode** – CPU switches back from kernel mode to user mode.
- 7) **Resume Application** – The application continues with the system call result.

System Calls

Examples of System Calls

Type of System Call	Windows	Linux
Process Control	ExitProcess()	exit()
	CreateProcess()	fork()
	WaitForSingleObject()	wait()
File Management	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()

System Calls

Examples of System Calls

Type of System Call	Windows	Linux
Device Management	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	SetTimer()	alarm()
	GetCurrentProcessID()	getpid()
	Sleep()	sleep()

System Calls

Examples of System Calls

Type of System Call	Windows	Linux
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()

Linux Commands & Shell Basics

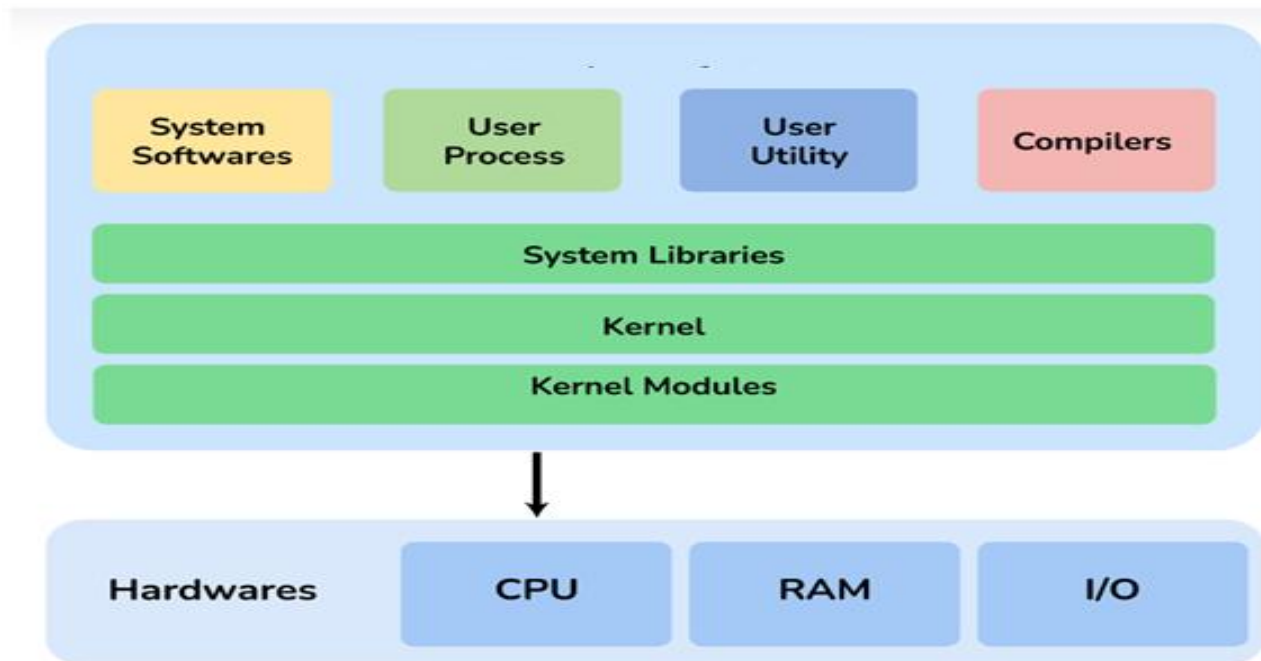


Introduction to Linux

- Linux is a **free and open-source operating system** based on UNIX.
- It powers everything from **smartphones (Android) to servers, supercomputers, and embedded systems.**
 - Developed by **Linus Torvalds in 1991.**
 - Uses a **command-line interface (CLI)** through a shell (like **bash, zsh**, etc.).
 - Known for **security, stability, and flexibility.**
- The **shell** is a program that takes user commands and passes them to the OS.

Linux Architecture

- Linux follows a **modular architecture** that efficiently manages system resources and processes.
- It consists of **several layers**, each responsible for specific tasks, ensuring stability, security, and performance.



Linux Architecture

Key Components of Linux Architecture:

- **User Space**
 - **System Software, User Processes, Utilities, Compilers:** Interact with system libraries and the kernel to perform tasks.
- **System Libraries**
 - Provide standard functions and APIs used by applications to interact with the kernel.
- **Kernel**
 - Core part of the OS that manages CPU, memory, and devices. Bridges user space and hardware.
- **Kernel Modules**
 - Loadable parts of the kernel for adding functionality (e.g., device drivers).
- **Hardware Layer**
 - CPU, RAM, I/O Devices: Controlled and accessed through the kernel.

Linux Commands & Shell Basics

Linux Basic Commands

S.No	Syntax	Description and Output
1	pwd	Print Working Directory Displays the full path name
2	whoami	Displays your logged in user id
3	cd / cd <target> cd ~	Change directory to the root of the filesystem Change directory to “target” directory Change directory to your home directory
4	mkdir my_directory	Create the directory “my_directory”
5	rm <file> rm -r <directory > rm -f <filename> rm -rf <directory > rm -i <file / directory>	Removes a specific file Removes a directory Removes a specific file - ignore nonexistent files and arguments, never prompt Removes a directory, ignore nonexistent directory and arguments, never prompt Prompt before Removal of file /directory

Linux Commands & Shell Basics

Linux Basic Commands

S.No	Syntax	Description and Output
6	ls ls -l ls -la	Directory listing Long listing, displays file ownership Displays hidden files/directories
7	cp source_file target_file cp -r ~/source_dir ~/target_dir	Copies a specific file Copies all files and sub-dirs
8	mv [source_file] [target_file]	Move file or directory
9	head file_name	Displays the first 10 lines of the file
10	tail file_name	Displays the last 10 lines of the file
11	touch <options><file/directory name>	Allows us to update the timestamps on existing files and directories as well as creating new, empty files.

Linux Commands & Shell Basics

Linux Basic Commands

S.No	Syntax	Description and Output
12	<code>cat > testsample.txt</code> <code>cat -n file.txt</code> <code>cat count.txt more</code> <code>cat count.txt > testsample.txt</code> <code>cat filesource >> appendfile</code>	Creates a file Displays the contents of file.txt to standard out with line number Displays the contents of the file with forward paging Copy the contents of one file to another file Append the contents of one file to the end of another file.
13	<code>chmod octal_value file_name</code>	Change the permissions of file to octal, which can be found separately for Owner, group, and others by adding: • 4 – read (r) • 2 – write (w) • 1 – execute (x) <code>chmod 777</code> – read, write, execute for all <code>chmod 755</code> – rwx for owner, rx for group and others.

Linux Commands & Shell Basics

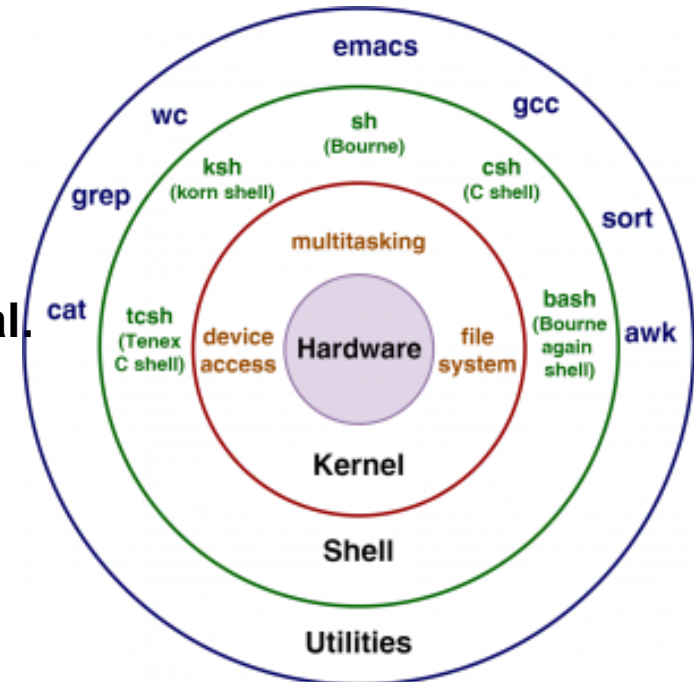
Linux Basic Commands

S.No	Syntax	Description and Output
14	grep pattern files	Search for pattern in files Pattern - A string that specifies the pattern of the Regular Expression
15	man ls	Show manual/help for a command
16	ps aux	Show active processes
17	echo	Print text to terminal

Linux Commands & Shell Basics

Shell

- A Shell is a **command-line interface (CLI)** program that allows users to communicate with the **operating system's kernel**.
- It interprets the **commands** entered by the user and converts them into instructions for the OS to execute.
- The shell **is like a tool to run Linux commands**.
- The shell gets started when the **user logs in or starts the terminal**.



Linux Commands & Shell Basics

Shell Types

- There are several types of shells available in Unix/Linux systems, each offering **different features, user experiences, and scripting capabilities.**

Most commonly used ones are:

- **BASH (Bourne Again SHell)** - Widely used shell and default Login Shell in Linux Systems
- **CSH (C Shell)** – C Shell's syntax and usage are similar to C Programming Language
- **KSH (Korn Shell)** -Korn Shell also was the base for the POSIX Shell standard specifications etc.

Functions of Shell

- **User Interface:** The shell provides a way for **users to interact** with the operating system through commands.
- **Program Execution:** It allows users to **run, manage, and control software applications**.
- **Automation:** Shell scripts automate **repetitive tasks**, saving time and effort.
- **System Management:** System administrators use the shell for **configuring and maintaining systems**.
- **Remote Access:** Shells enable secure **control of remote systems** via tools like SSH.

Advantages & Disadvantages of Shell

Advantages of Shell:

- **Efficient & Lightweight:** Faster than GUIs and uses fewer system resources.
- **Powerful & Flexible:** Offers advanced control and integrates well with tools like grep, awk, and cron.
- **Automatable & Remote-Friendly:** Supports scripting for automation and remote system management via SSH.

Disadvantages of Shell:

- **Hard to Learn:** Requires memorizing commands and syntax, which can be tough for beginners.
- **Error-Prone:** Small mistakes (like typos) can lead to serious system issues.
- **Non-Visual & Platform-Specific:** No graphical interface and some scripts may not work across different systems or shells.

THANK YOU